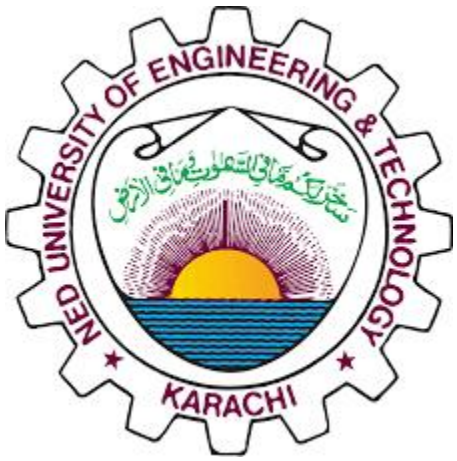

NED University of Engineering and Technology

"Design and Implementation of a Verilog-based Round-Robin Arbiter with Fixed-Time Slice Allocation."

OPEN ENDED LAB
VLSI (EL-408)

Course Instructor: Ms. Mariam Jamshaid

Group Members:

Syeda Ayesha Ali	EL-22008
Faraz Malik Awan	EL-22049
Zukhruf Khan	EL-22059
Tamseela Hussain	EL-22062

Objective

The objective of this project is to design, implement, and simulate a 4-input Round-Robin Arbiter using Verilog HDL. The arbiter aims to achieve fair and synchronized resource allocation among multiple requests by assigning each one a fixed time slice for access. Students will demonstrate an understanding of round-robin scheduling, time slice allocation, and one-hot grant generation in digital systems.

Abstract

In digital systems, multiple devices or modules often compete for access to a shared resource such as a bus or memory. An arbiter manages these requests to ensure orderly and conflict-free access.
This project implements a Verilog-based Round-Robin Arbiter that allocates access fairly among four requesters (REQ0–REQ3). Using a round-robin scheduling algorithm, each request is granted access for a fixed number of clock cycles, known as the time slice.

The arbiter outputs a one-hot encoded grant signal, ensuring that only one request is active at a time. After each time slice, the arbiter moves to the next request in sequence, maintaining fairness and preventing starvation. The design is fully synchronous and verified through simulation in ModelSim. The waveform confirms that the arbiter correctly cycles through all requests with the specified time slice duration.

Literature Review

Resource sharing and arbitration mechanisms are essential in modern VLSI and digital systems to manage concurrent access efficiently.
Traditional arbiter designs such as fixed-priority arbiters can lead to starvation of lower-priority requests. To overcome this limitation, round-robin scheduling is widely used, where priority rotates cyclically among all requesters, ensuring fairness.

In hardware design, round-robin arbiters are implemented using counters, timers, and combinational logic that sequentially grant access to multiple clients. This approach provides deterministic timing, predictable performance, and is particularly useful in multiprocessor, memory, and communication bus systems.
The use of fixed-time slice allocation further enhances control by guaranteeing each requester a defined service period before the next one is served.

Block Diagram

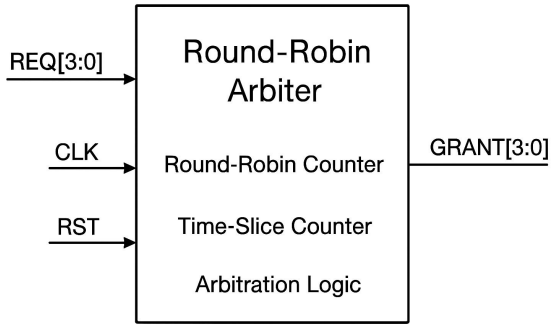

Figure 1: Block Diagram

Block Explanation

- **REQ[3:0]:** Four input request signals from different modules.
- **CLK:** Clock input that synchronizes all operations.
- **RST:** Reset input to initialize the system to a known state.
- **Arbiter Block:** Contains three submodules
    - *Round-Robin Counter* for rotating priority order
    - *Time Slice Counter* for tracking duration per request
    - *Arbitration Logic* for generating one-hot grants
- **GRANT[3:0]:** One-hot output signal representing which request currently has access.

Design Explanation

The system is designed to allocate access to four input requests (REQ0–REQ3) in a round-robin order. It consists of three main components: a round-robin counter, a time slice counter, and arbitration logic.

Design Choices

The following design decisions were made to ensure simplicity, clarity, and functionality:

- **Number of Requests (N = 4):** Chosen for a 4-input arbiter as per lab requirement.
- **Time Slice (TIME_SLICE = 4):** Each request retains the grant for four clock cycles to demonstrate fixed scheduling.
- **One-Hot Grant Scheme:** Ensures only one request is active at any time, preventing bus conflicts.
- **Synchronous Design:** All signals are clock-driven for stability and deterministic behavior.

- **Parameterization:** Parameters like `N` and `TIME_SLICE` allow easy scaling to more requests or different slice durations.

Working of the Circuit

When the circuit starts and reset is active, the arbiter initializes the system by granting access to REQ0. After the reset signal goes low, the round-robin counter begins cycling through all four requests in order. Each request receives a fixed time slice of four clock cycles, controlled by the time slice counter. After the time slice completes, the arbiter moves to the next request and updates the one-hot grant output accordingly.
This process repeats continuously (REQ0 → REQ1 → REQ2 → REQ3 → REQ0), ensuring that every request gets equal and fair access without overlap or starvation.

State Transition

The arbiter operates based on sequential state transitions controlled by the clock.
Each state corresponds to one active grant output.

- State 0: Grant = 0001 (REQ0 active)
- State 1: Grant = 0010 (REQ1 active)
- State 2: Grant = 0100 (REQ2 active)
- State 3: Grant = 1000 (REQ3 active)

After each fixed time slice (4 clock cycles), the system transitions to the next state in a circular manner. When reset is asserted, the state returns to State 0.
This ensures predictable and fair scheduling of all requests.

Implementation Logic

The implementation combines a counter, arbiter logic, and time-slice control:

- Round-Robin Counter — Cycles through request indices (0–3).
- Time Counter — Counts clock cycles within each time slice.
- Arbiter Logic — Checks request validity and activates the corresponding one-hot grant.

After the counter completes the slice, the current index is incremented, and the next grant is activated. The design runs continuously, producing a rotating one-hot grant output that matches round-robin scheduling.

Module Explanation

The design is composed of two main Verilog modules and a testbench:

1. Round-Robin Counter
    - A 2-bit counter that cycles through request indices (0–3) in round-robin order.
2. Arbiter Logic
    - Uses the counter output and request lines to decide which request gets access.
    - Generates one-hot encoded grant signals.
    - Implements a fixed time slice counter to maintain each grant for a set duration.
3. Time Slice Control
    - The TIME_SLICE parameter (set to 4) defines how many clock cycles each request retains its grant.
    - After four cycles, the arbiter moves to the next request.

During operation, the arbiter:

- Starts from REQ0 after reset.
- Grants access to each requester in sequence (REQ0 → REQ1 → REQ2 → REQ3).
- Each grant lasts for four clock cycles.
- Once all requests are served, the process repeats continuously.

This design ensures fairness, synchronization, and predictability in resource allocation.

Verilog Code

round_robin_counter.v

```
// Round Robin Counter Module
// Counts from 0 to N-1 and repeats
module round_robin_counter #(parameter N = 4) (
    input clk,              // Clock input
    input rst,              // Reset input
    output reg [1:0] count  // 2-bit counter output
);
    // Count increases on each clock edge, resets when rst = 1
    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 0;             // Reset count to 0
        else
            count <= (count + 1) % N; // Increment count, wrap after N
    end
endmodule
```

round_robin_arbiter.v

```
// Round Robin Arbiter Module
module round_robin_arbiter #(parameter N = 4, parameter TIME_SLICE = 4)(
    input clk,                  // Clock input
    input rst,                  // Active-high reset
    input [N-1:0] req,          // N request inputs
    output reg [N-1:0] grant    // One-hot grant output
);

    reg [1:0] current;          // Current active request index
```

```
reg [2:0] time_count;          // Time slice counter

// Round-robin arbitration process
always @(posedge clk or posedge rst) begin
    if (rst) begin
        // Initialize on reset
        current <= 0;
        time_count <= 0;
        grant <= 4'b0001;      // Start with REQ0
    end
    else begin
        // If current request is active and time slice not done
        if (req[current] && time_count < TIME_SLICE - 1) begin
            time_count <= time_count + 1;   // Continue serving same request
            grant <= (4'b0001 << current);  // Keep same grant active
        end
        else begin
            // Move to next active request
            time_count <= 0;
            current <= (current + 1) % N;

            // Find the next active request
            if (req[(current + 1) % N])
                grant <= (4'b0001 << ((current + 1) % N));
            else if (req[(current + 2) % N])
                grant <= (4'b0001 << ((current + 2) % N));
            else if (req[(current + 3) % N])
                grant <= (4'b0001 << ((current + 3) % N));
            else
                grant <= 0; // No requests active
```

```
            end
        end
    end
endmodule
```

tb_round_robin_arbiter.v (Testbench)

```
`timescale 1ns/1ps
// Testbench for Round Robin Arbiter
module tb_round_robin_arbiter();

    reg clk, rst;          // Clock and reset
    reg [3:0] req;         // Request inputs
    wire [3:0] grant;      // Grant outputs

    // Instantiate the arbiter module
    round_robin_arbiter uut(
        .clk(clk),
        .rst(rst),
        .req(req),
        .grant(grant)
    );

    // Generate clock (10ns period)
    initial begin
        clk = 0;
        forever #5 clk = ~clk;   // Toggle every 5ns
    end

    // Apply test cases
    initial begin
        rst = 1; req = 4'b0000;  // Apply reset
```

```
        #10;
        rst = 0;                // Release reset
        req = 4'b1111;          // All requests active
        #200;
        req = 4'b1001;          // Only Req0 and Req3 active
        #100;
        req = 4'b0100;          // Only Req2 active
        #100;
        $stop;                  // Stop simulation
    end
endmodule
```

Simulation Results & Waveforms

The simulation of the Round-Robin Arbiter was carried out using ModelSim.
The waveform verifies that the arbiter generates one-hot grant outputs (0001 → 0010 → 0100 → 1000) in a cyclic order.
Each grant remains high for four clock cycles, confirming the correct implementation of the fixed time slice.
When all requests are active, the arbiter fairly rotates the grant among them.
When some requests are inactive, the arbiter skips them and serves the next available request.

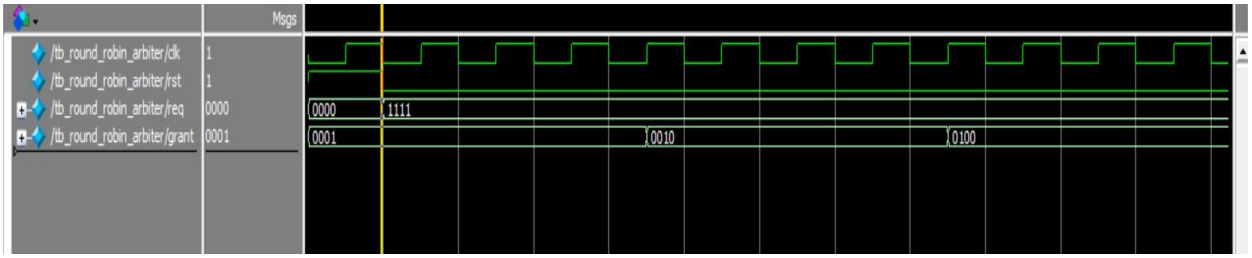

Figure 2:waveform showing system initialization after reset

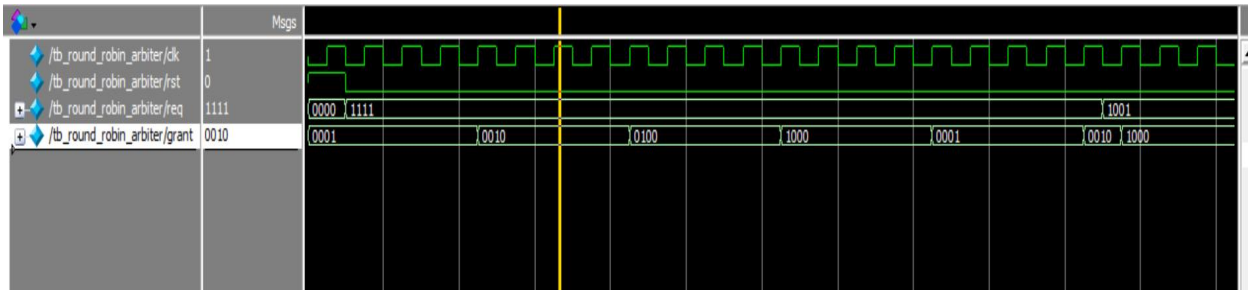

Figure 3:waveform showing round-robin grant rotation (REQ0 → REQ1 → REQ2 → REQ3)

Figure 4:waveform showing the arbiter skipping inactive requests.

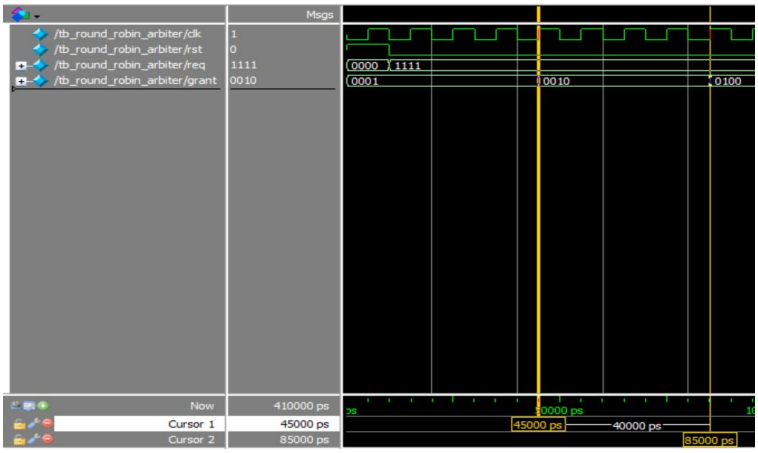

Figure 5:Measurement of time slice duration. Each grant remains active for 4 clock cycles (≈40 ns = 40,000 ps)

The waveform confirms that the arbiter provides one-hot grant outputs in a round-robin sequence. Each grant stays active for the defined time slice of four clock cycles. The design behaves correctly during reset, under all requests active, and when only specific requests are enabled. The simulation results match the expected functionality of the round-robin arbiter.

Conclusion

The designed Round-Robin Arbiter with Fixed-Time Slice Allocation successfully achieves fair, cyclic, and synchronized access control among four input requests.
Simulation results verify that each request receives access for exactly four clock cycles in a rotating one-hot sequence.

This design demonstrates practical understanding of scheduling algorithms, Verilog HDL design, and digital system synchronization.
The project fulfills the Open-Ended Lab objective by implementing a functional hardware scheduling mechanism that can be extended for bus arbitration or multiprocessor systems.